

# Assignment - 3

## Analyze Sales Data

Name: Suyash Saxena

Dataset: Sample Superstore

Database: SQLite

### Create Customer Table

```
-- Create Customers Table
CREATE TABLE customers AS
SELECT DISTINCT
"Customer ID" AS customer_id,
"Customer Name" AS customer_name,
Segment
FROM superstore_raw;

SELECT * FROM customers;
```

|   | customer_id | customer_name   | Segment     |
|---|-------------|-----------------|-------------|
| 1 | CG-12520    | Claire Gute     | Consumer    |
| 2 | DV-13045    | Darrin Van Huff | Corporate   |
| 3 | SO-20335    | Sean O'Donnell  | Consumer    |
| 4 | BH-11710    | Brosina Hoffman | Consumer    |
| 5 | AA-10480    | Andrew Allen    | Consumer    |
| 6 | IM-15070    | Irene Maddox    | Consumer    |
| 7 | HP-14815    | Harold Pawlan   | Home Office |
| 8 | PK-19075    | Pete Kriz       | Consumer    |

### Create Products Table

```
-- Create Products Table
CREATE TABLE products AS
SELECT DISTINCT
"Product ID" AS product_id,
"Product Name" AS product_name,
Category,
"Sub-Category" AS sub_category
FROM superstore_raw;

SELECT * FROM products;
```

|   | product_id      | product_name                              | Category        | sub_category |
|---|-----------------|-------------------------------------------|-----------------|--------------|
| 1 | FUR-B0-10001798 | Bush Somerset Collection Bookcase         | Furniture       | Bookcases    |
|   | FUR-CH-10000454 | Hon Deluxe Fabric Upholstered Stacking... | Furniture       | Chairs       |
| 3 | OFF-LA-10000240 | Self-Adhesive Address Labels for Typew... | Office Supplies | Labels       |
| 4 | FUR-TA-10000577 | Bretford CR4500 Series Slim Rectangula... | Furniture       | Tables       |
| 5 | OFF-ST-10000760 | Eldon Fold 'N Roll Cart System            | Office Supplies | Storage      |
|   | FUR-FU-10001487 | Eldon Expressions Wood and Plastic Des... | Furniture       | Furnishings  |
| 7 | OFF-AR-10002833 | Newell 322                                | Office Supplies | Art          |
| 8 | TEC-PH-10002275 | Mitel 5320 IP Phone VoIP phone            | Technology      | Phones       |

## Create Orders Table

```
-- Create Orders Table
CREATE TABLE orders AS
SELECT
"Order ID" AS order_id,
"Customer ID" AS customer_id,
"Product ID" AS product_id,
Sales,
Quantity,
Profit,
Region,
"Order Date" AS order_date
FROM superstore_raw;

SELECT * FROM orders;
```

|   | order_id       | customer_id | product_id      | Sales    | Quantity | Profit   | Region | order_date |
|---|----------------|-------------|-----------------|----------|----------|----------|--------|------------|
| 1 | CA-2016-152156 | CG-12520    | FUR-B0-10001798 | 261.96   | 2        | 41.9136  | South  | 11/8/2016  |
| 2 | CA-2016-152156 | CG-12520    | FUR-CH-10000454 | 731.94   | 3        | 219.582  | South  | 11/8/2016  |
| 3 | CA-2016-138688 | DV-13045    | OFF-LA-10000240 | 14.62    | 2        | 6.8714   | West   | 6/12/2016  |
| 4 | US-2015-108966 | SO-20335    | FUR-TA-10000577 | 957.5775 | 5        | -383.031 | South  | 10/11/2015 |
| 5 | US-2015-108966 | SO-20335    | OFF-ST-10000760 | 22.368   | 2        | 2.5164   | South  | 10/11/2015 |
| 6 | CA-2014-115812 | BH-11710    | FUR-FU-10001487 | 48.86    | 7        | 14.1694  | West   | 6/9/2014   |
| 7 | CA-2014-115812 | BH-11710    | OFF-AR-10002833 | 7.28     | 4        | 1.9656   | West   | 6/9/2014   |
| 8 | CA-2014-115812 | BH-11710    | TEC-PH-10002275 | 907.152  | 6        | 90.7152  | West   | 6/9/2014   |

## Step 2: Perform Required Queries

Write and execute SQL queries for each of the following:

1. Find all orders where sales are greater than the average sales. (*Subquery*)

```
-- 1. Find all orders where sales are greater than the average sales
SELECT *
FROM orders
WHERE sales >
(SELECT AVG(sales)
FROM orders);
```

|   | order_id       | customer_id | product_id      | Sales    | Quantity | Profit   | Region  | order_date |
|---|----------------|-------------|-----------------|----------|----------|----------|---------|------------|
| 1 | CA-2016-152156 | CG-12520    | FUR-B0-10001798 | 261.96   | 2        | 41.9136  | South   | 11/8/2016  |
| 2 | CA-2016-152156 | CG-12520    | FUR-CH-10000454 | 731.94   | 3        | 219.582  | South   | 11/8/2016  |
| 3 | US-2015-108966 | SO-20335    | FUR-TA-10000577 | 957.5775 | 5        | -383.031 | South   | 10/11/2015 |
| 4 | CA-2014-115812 | BH-11710    | TEC-PH-10002275 | 907.152  | 6        | 90.7152  | West    | 6/9/2014   |
| 5 | CA-2014-115812 | BH-11710    | FUR-TA-10001539 | 1706.184 | 9        | 85.3092  | West    | 6/9/2014   |
| 6 | CA-2014-115812 | BH-11710    | TEC-PH-10002033 | 911.424  | 4        | 68.3568  | West    | 6/9/2014   |
| 7 | CA-2016-161389 | IM-15070    | OFF-BI-10003656 | 407.976  | 3        | 132.5922 | West    | 12/5/2016  |
| 8 | CA-2014-105893 | PK-19075    | OFF-ST-10004186 | 665.88   | 6        | 13.3176  | Central | 11/11/2014 |

2. Find the highest sales order for each customer. (*Subquery*)

```
-- 2. Find the highest sales order for each customer.
SELECT *
FROM orders
WHERE sales =
(SELECT MAX(sales)
FROM orders
WHERE customer_id = orders.customer_id);
```

|   | order_id       | customer_id | product_id      | Sales    | Quantity | Profit     | Region | order_date |
|---|----------------|-------------|-----------------|----------|----------|------------|--------|------------|
| 1 | CA-2014-145317 | SM-20320    | TEC-MA-10002412 | 22638.48 | 6        | -1811.0784 | South  | 3/18/2014  |

3. Calculate total sales for each customer. (CTE)

```
-- 3. Calculate total sales for each customer.
WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
customer_id,
total_sales
FROM sales_summary
ORDER BY total_sales DESC;
```

|   | customer_id | total_sales        |
|---|-------------|--------------------|
| 1 | SM-20320    | 25043.05           |
| 2 | TC-20980    | 19052.218          |
| 3 | RB-19360    | 15117.339          |
| 4 | TA-21385    | 14595.62           |
| 5 | AB-10105    | 14473.571          |
| 6 | KL-16645    | 14175.229          |
| 7 | SC-20095    | 14142.334          |
| 8 | HL-15040    | 12873.297999999999 |

4. Find customers whose total sales are above average. (CTE + Subquery)

```
-- 4. Find customers whose total sales are above average
WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
customer_id,
total_sales
FROM sales_summary
WHERE total_sales >
(SELECT AVG(total_sales)
FROM sales_summary)
ORDER BY total_sales DESC;
```

|    | customer_id | total_sales |
|----|-------------|-------------|
| 31 | DR-12940    | 8350.868    |
| 32 | NF-18385    | 8322.826    |
| 33 | KD-16270    | 8282.358    |
| 34 | NC-18535    | 8241.739    |
| 35 | HM-14860    | 8236.7648   |
| 36 | KD-16495    | 8181.256    |
| 37 | SB-20290    | 8057.891    |
| 38 | ZC-21910    | 8025.707    |

294 rows

5. Rank all customers based on total sales. (*Window Function*)

```
-- 5. Rank all customers based on total sales.
WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
customer_id,
total_sales,
RANK() OVER (ORDER BY total_sales DESC) AS rank_position
FROM sales_summary
ORDER BY rank_position;
```

|   | customer_id | total_sales        | rank_position |
|---|-------------|--------------------|---------------|
| 1 | SM-20320    | 25043.05           | 1             |
| 2 | TC-20980    | 19052.218          | 2             |
| 3 | RB-19360    | 15117.339          | 3             |
| 4 | TA-21385    | 14595.62           | 4             |
| 5 | AB-10105    | 14473.571          | 5             |
| 6 | KL-16645    | 14175.229          | 6             |
| 7 | SC-20095    | 14142.334          | 7             |
| 8 | HL-15040    | 12873.297999999999 | 8             |

6. Assign row numbers to each order within a customer. (*Window Function + PARTITION BY*)

```
-- 6. Assign row numbers to each order within a customer.
SELECT
customer_id,
order_id,
sales,
ROW_NUMBER() OVER
(PARTITION BY customer_id
ORDER BY sales DESC) AS row_number
FROM orders
ORDER BY customer_id, row_number;
```

|   | customer_id | order_id       | Sales    | row_number |
|---|-------------|----------------|----------|------------|
| 1 | AA-10315    | CA-2016-103982 | 3930.072 | 1          |
| 2 | AA-10315    | CA-2014-128055 | 673.568  | 2          |
| 3 | AA-10315    | CA-2016-103982 | 431.976  | 3          |
| 4 | AA-10315    | CA-2017-147039 | 362.94   | 4          |
| 5 | AA-10315    | CA-2014-128055 | 52.98    | 5          |
| 6 | AA-10315    | CA-2016-103982 | 41.72    | 6          |
| 7 | AA-10315    | CA-2015-121391 | 26.96    | 7          |
| 8 | AA-10315    | CA-2014-138100 | 14.94    | 8          |

7. Display top 3 customers based on total sales. (*Window Function*)

```
-- 7. Display top 3 customers based on total sales.
WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id),
customer_ranking AS
(SELECT
customer_id,
total_sales,
RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM sales_summary)

SELECT
customer_id,
total_sales,
sales_rank
FROM customer_ranking
WHERE sales_rank <= 3;
```

|   | customer_id | total_sales | sales_rank |
|---|-------------|-------------|------------|
| 1 | SM-20320    | 25043.05    | 1          |
| 2 | TC-20980    | 19052.218   | 2          |
| 3 | RB-19360    | 15117.339   | 3          |

### Step 3: Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

```
WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
cust.customer_name,
ss.total_sales,
RANK() OVER (ORDER BY ss.total_sales DESC) AS customer_rank
FROM sales_summary ss
JOIN customers cust
ON ss.customer_id = cust.customer_id
ORDER BY customer_rank;
```

|   | customer_name | total_sales        | customer_rank |
|---|---------------|--------------------|---------------|
| 1 | Sean Miller   | 25043.05           | 1             |
| 2 | Tamara Chand  | 19052.218          | 2             |
| 3 | Raymond Buch  | 15117.339          | 3             |
| 4 | Tom Ashbrook  | 14595.62           | 4             |
| 5 | Adrian Barton | 14473.571          | 5             |
| 6 | Ken Lonsdale  | 14175.229          | 6             |
| 7 | Sanjit Chand  | 14142.334          | 7             |
| 8 | Hunter Lopez  | 12873.297999999999 | 8             |

## Mini Project: Customer Sales Insights

1. Who are the top 5 customers?

```
-- 1. Top 5 Customers

WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
cust.customer_name,
ss.total_sales
FROM sales_summary ss
JOIN customers cust
ON ss.customer_id = cust.customer_id
ORDER BY ss.total_sales DESC
LIMIT 5;
```

|   | customer_name | total_sales |
|---|---------------|-------------|
| 1 | Sean Miller   | 25043.05    |
| 2 | Tamara Chand  | 19052.218   |
| 3 | Raymond Buch  | 15117.339   |
| 4 | Tom Ashbrook  | 14595.62    |
| 5 | Adrian Barton | 14473.571   |

5 rows

2. Who are the bottom 5 customers?

```
-- 2. Bottom 5 Customers

WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
cust.customer_name,
ss.total_sales
FROM sales_summary ss
JOIN customers cust
ON ss.customer_id = cust.customer_id
ORDER BY ss.total_sales ASC
LIMIT 5;
```

|   | customer_name   | total_sales |
|---|-----------------|-------------|
| 1 | Thais Sissman   | 4.833       |
| 2 | Lela Donovan    | 5.304       |
| 3 | Carl Jackson    | 16.52       |
| 4 | Mitch Gastineau | 16.739      |
| 5 | Roy Skaria      | 22.328      |

5 rows



3. Which customers made only one order?

```
SELECT
cust.customer_name,
COUNT(ord.order_id) AS total_orders
FROM orders ord
JOIN customers cust
ON ord.customer_id = cust.customer_id
GROUP BY ord.customer_id, cust.customer_name
HAVING COUNT(ord.order_id) = 1;
```

|   | customer_name     | total_orders |
|---|-------------------|--------------|
| 1 | Anthony O'Donnell | 1            |
| 2 | Carl Jackson      | 1            |
| 3 | Jocasta Rupert    | 1            |
| 4 | Lela Donovan      | 1            |
| 5 | Ricardo Emerson   | 1            |

4. Which customers have above-average sales?

```
WITH sales_summary AS
(SELECT
customer_id,
SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id)

SELECT
cust.customer_name,
ss.total_sales
FROM sales_summary ss
JOIN customers cust
ON ss.customer_id = cust.customer_id
WHERE ss.total_sales >
(SELECT AVG(total_sales)
FROM sales_summary)
ORDER BY ss.total_sales DESC;
```

|   | customer_name | total_sales        |
|---|---------------|--------------------|
| 1 | Sean Miller   | 25043.05           |
| 2 | Tamara Chand  | 19052.218          |
| 3 | Raymond Buch  | 15117.339          |
| 4 | Tom Ashbrook  | 14595.62           |
| 5 | Adrian Barton | 14473.571          |
| 6 | Ken Lonsdale  | 14175.229          |
| 7 | Sanjit Chand  | 14142.334          |
| 8 | Hunter Lopez  | 12873.297999999999 |

5. What is the highest order value per customer?

```
SELECT
cust.customer_name,
MAX(ord.sales) AS highest_order_value
FROM orders ord
JOIN customers cust
ON ord.customer_id = cust.customer_id
GROUP BY ord.customer_id, cust.customer_name
ORDER BY highest_order_value DESC;
```

|   | customer_name | highest_order_value |
|---|---------------|---------------------|
| 1 | Sean Miller   | 22638.48            |
| 2 | Tamara Chand  | 17499.95            |
| 3 | Raymond Buch  | 13999.96            |
| 4 | Tom Ashbrook  | 11199.968           |
| 5 | Hunter Lopez  | 10499.97            |
| 6 | Adrian Barton | 9892.74             |
| 7 | Sanjit Chand  | 9449.95             |
| 8 | Bill Shonely  | 9099.93             |